

Lab : Mastering Multilayer Perceptron

Overview: Multi-Layer Perceptron (MLP) for Classification and Approximation

Multi-Layer Perceptrons (MLPs) are a fundamental type of artificial neural network commonly used for solving problems in both classification and function approximation. They belong to the class of feedforward neural networks and consist of an input layer, one or more hidden layers, and an output layer. MLPs are capable of learning complex patterns and relationships from data through the process of backpropagation and gradient descent. In this session you have to work on two problems:

1. Lab 1: Multi-Class Classification using scikit-learn MLP Classifier on Forest Cover Type Dataset.
2. Lab 2: Studying Universal Approximation in Neural Networks.

Lab 1: Multi-Class Classification using scikit-learn MLP Classifier on Forest Cover Type Dataset

Objective

In this lab, you will implement a **Multi-Layer Perceptron (MLP)** classifier using **scikit-learn** to classify forest cover types. You have already performed the data profiling and preprocessing steps in a previous lab, and you will now focus on building and evaluating the MLP model.

Recap: Previous Lab

- You have already preprocessed the **Forest Cover Type Dataset**:
 - **Feature scaling** using StandardScaler.
 - **One-hot encoding** for the target variable.
 - **Train-test split** (80% for training and 20% for testing).

Now, we'll proceed to use **scikit-learn's MLPClassifier** to build the MLP model.

Build the MLP Classifier using scikit-learn

1. **Import the MLPClassifier:** We'll use the **MLPClassifier** from **scikit-learn** to build the multi-layer perceptron model.

```
from sklearn.neural_network import MLPClassifier
```

2. **Define the MLP Model:** Define the architecture of the MLPClassifier. You can specify parameters like the number of hidden layers, activation functions, and optimization settings. We'll start with 2 hidden layers of 128 and 64 neurons, using ReLU activation and the Adam optimizer.

```
# Define the MLP model with two hidden layers (128, 64 neurons)
mlp = MLPClassifier(hidden_layer_sizes=(128, 64), activation='relu', solver='adam',
max_iter=200, random_state=42)
```

3. **Train the MLP Model:** Fit the model on the training dataset. Since MLPClassifier doesn't require one-hot encoding for the target labels, you can use the original integer-encoded labels.

```
# Train the model
mlp.fit(X_train, y_train.argmax(axis=1)) # Using integer-encoded labels for training
```

4. **Evaluate the Model:** After training, evaluate the model on the test set. We'll predict the class labels for the test set and generate evaluation metrics like accuracy, confusion matrix, and classification report.

```
# Make predictions
y_pred_mlp = mlp.predict(X_test)
# Evaluate the model
from sklearn.metrics import classification_report, confusion_matrix
# Print classification report
print('Classification Report (MLP Classifier):\n',
classification_report(y_test.argmax(axis=1), y_pred_mlp))
# Confusion matrix
cm_mlp = confusion_matrix(y_test.argmax(axis=1), y_pred_mlp)
print('Confusion Matrix (MLP Classifier):\n', cm_mlp)
```

5. **Visualize the Confusion Matrix:** Visualize the confusion matrix as a heatmap to better understand where the model performed well and where it struggled.

```
import seaborn as sns
import matplotlib.pyplot as plt
# Plot confusion matrix
plt.figure(figsize=(10,7))
sns.heatmap(cm_mlp, annot=True, fmt='d', cmap='Blues', xticklabels=np.arange(1, 8),
yticklabels=np.arange(1, 8))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix (MLP Classifier)')
plt.show()
```

6. **Experiment with Hyperparameters:**

- Try to tune hyperparameters such as the number of hidden layers, neurons, learning rate (learning_rate_init), and regularization (alpha).
- Example of adjusting learning rate and regularization:

```
# Define a new MLP model with different hyperparameters
mlp_tuned = MLPClassifier(hidden_layer_sizes=(128, 64), activation='relu',
solver='adam', max_iter=200, alpha=0.001, learning_rate_init=0.001, random_state=42)
# Train the tuned model
mlp_tuned.fit(X_train, y_train.argmax(axis=1))
# Predict and evaluate the tuned model
y_pred_tuned = mlp_tuned.predict(X_test)
print('Classification Report (Tuned MLP Classifier):\n',
classification_report(y_test.argmax(axis=1), y_pred_tuned))
```

Analysis and Discussion

1. **Compare Performance:** Compare the results from the **MLPClassifier** with previous models (e.g., Logistic Regression or k-NN). Discuss which model performed better for the dataset and why.
2. **Experiment with Hyperparameters:**
 - Adjust the number of hidden layers, learning rate, and regularization to see how they impact the model's performance.
 - You can also experiment with different solvers such as sgd for gradient descent.
3. **Training Time:**
 - Note that MLPClassifier can take more time to train compared to other models like Logistic Regression or k-NN. Discuss trade-offs between accuracy and computation time.

Bonus Challenges

- **Early Stopping:** Enable early stopping to prevent overfitting by adding early_stopping=True to the MLPClassifier.
- **Learning Curves:** Plot the loss curve to understand how well the model converged.
- **Cross-Validation:** Perform k-fold cross-validation to get a more robust estimate of the model's performance.

```
from sklearn.model_selection import cross_val_score
# Perform 5-fold cross-validation
cv_scores = cross_val_score(mlp, X_train, y_train.argmax(axis=1), cv=5)
print("Cross-Validation Accuracy: %.2f%%" % (cv_scores.mean()*100))
```

Lab 2: Studying Universal Approximation in Neural Networks

Objective

The objective of this lab is to study a fundamental property of static neural networks (non-recurrent): sparse approximation. We will do this by training a Multi-Layer Perceptron (MLP) with one hidden layer on a piecewise-defined function and analyze how the network approximates the function.

Step 1: Data Generation

1. Define the function ($f(x)$) as follows:

$$f(x) = \begin{cases} \sin(\pi x) & \text{if } x \in]-1, 1[\\ 0 & \text{if } x \in [-2, -1] \cup [1, 2] \end{cases}$$

2. Add noise $\mathcal{N}(0, 0.2)$ to the data.

3. Generate multiple training and test samples using this function.

You can use the following Python code to generate the data:

```
import numpy as np

# Function definition
def f(x):
    if -1 < x < 1:
        return np.sin(np.pi * x)
    else:
        return 0

# Generate data
np.random.seed(42)
x_train = np.random.uniform(-2, 2, 1000)
y_train = np.array([f(x) + np.random.normal(0, 0.2) for x in x_train])
```

```
# For test set
x_test = np.random.uniform(-2, 2, 200)
y_test = np.array([f(x) + np.random.normal(0, 0.2) for x in x_test])
```

Step 2: MLP Implementation with TensorFlow

In this step, we will train a simple MLP with one hidden layer on the generated data.

You can use the following Python code with TensorFlow to implement the MLP:

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Model definition
model = models.Sequential()
model.add(layers.Dense(10, activation='tanh', input_shape=(1,))) # Hidden layer with tanh
model.add(layers.Dense(1)) # Output layer with no activation function

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
model.fit(x_train, y_train, epochs=100, batch_size=32, validation_data=(x_test, y_test))
```

Recommendations

1. Start with a small number of neurons (e.g., 1, 3, 5, 7 neurons in the hidden layer) and observe the results.
2. Explore variations in the number of neurons in the hidden layer to see how sparse approximation changes with model capacity.
3. Test the generalization of the model by evaluating its performance on the test set.